ScreenLog — Movie and Television Tracker

Project Proposal

1. Project Title and Team Members

Project Title: ScreenLog — Movie and Television Tracker

Team Members: Jack Nusom, Medha Bhatnagar, Raif Rashid

2. Project Description and Motivation

ScreenLog is a full-stack web application for discovering movies and television shows, finding where to watch them, and keeping a personal viewing history with episode tracking, reviews, and recommendations.

Tracking entertainment across many streaming services is difficult: viewers forget which episode they last watched, lose recommendations, or spend time searching for where a title is available. ScreenLog brings this together in one browser-based application for casual viewers and movie or television enthusiasts, who keep watching on their existing services and use ScreenLog to manage their library and decide what to watch next.

3. Main Features

Core features (MVP). User accounts keep viewing history, preferences, and reviews available across sessions. The MVP includes five features:

1. **Search and Explore:** Users search movies and shows and open detail pages (poster, description, genres, release info, runtime) with a “Where to Watch” section of streaming, rental, and purchase options by country (default: United States).
2. **Personal Library and Viewing History:** Users add titles to a watchlist, mark movies watched, and label shows watching, completed, paused, or dropped; the app saves changes and supports filtering, updating, and removing entries.
3. **Episode Tracking:** Users mark episodes or whole seasons as watched (manual entry at first); the app calculates progress and identifies the next unwatched episode.
4. **Ratings and Reviews:** Users rate titles, write reviews, and read others’ feedback, and can edit or delete their own reviews; spoiler-marked reviews stay concealed until a reader chooses to reveal them.

5. **Personalized Recommendations:** The app suggests titles from viewing history, preferred genres, and ratings (highly rated titles count more), excluding watched movies and completed or dropped shows; new users pick favorite genres to get started.

Optional features (if time permits). Viewing statistics, upcoming-episode calendars, release reminders, and viewing-history import and export, after the five core features are complete.

4. Application Architecture

Front-end: Pages include Discover, movie and show details (title info, season and episode lists, community feedback, availability), a personal library, recommendations, and account settings. Users enter search terms, choose a country, manage watchlists, update episode progress, and submit ratings and reviews. The interface has loading, empty, and error states and supports desktop and mobile browsers.

Back-end: The server processes requests, validates input, authenticates users, and communicates with the database, ensuring users modify only their own library entries, viewing records, and reviews. It requests catalog data from The Movie Database (TMDB) and streaming availability from TMDB's JustWatch-powered endpoints, calculates episode progress and rating summaries, and generates recommendations with a simple genre-and-feedback scoring system.

Database: PostgreSQL permanently stores user accounts and preferences, library entries and viewing history, episode progress, and ratings and reviews; saved titles keep their external catalog IDs so activity stays tied to the right movie or show. Users create records by saving titles, logging viewing, and posting reviews; the server retrieves them for libraries and profiles; users update statuses, ratings, and progress or delete entries and reviews. All of it persists across browser sessions.

5. Technology and AI Tools

- **Front-end:** React with TypeScript, HTML, and CSS
- **Back-end:** Python with FastAPI
- **Database:** PostgreSQL
- **External data:** TMDB for movie and television metadata, images, and JustWatch-powered streaming availability

- **Development tools:** GitHub for version control; ChatGPT or Codex for planning, implementation, debugging, and tests

We will include the required TMDB and JustWatch attribution, present availability by country with links to TMDB's watch-options pages (per its API FAQ and watch-provider documentation), and review and verify all generated code so every member understands their contributions. These choices may be revised during development.

6. Development Plan and Team Responsibilities

- **Raif Rashid:** Front-end layout, search and detail pages, and personal library interface; connect these screens to the server and verify responsive behavior.
- **Medha Bhatnagar:** Authentication, database models, library and episode-tracking endpoints, and deployment configuration.
- **Jack Nusom:** Recommendation logic, ratings and review functionality with its interface components, and coordinating integration testing.

All members will join planning, code review, testing, and demo preparation. We will use a shared GitHub repository with feature branches and pull requests, agree on request and response formats before connecting components, and integrate working features throughout development.

Milestone	Planned work
Oct 30–Nov 5	Finalize scope, sketch screens, set up repository, verify external API access
Nov 6–15	Accounts, database persistence, catalog search, basic library management
Nov 16–25	Episode tracking, ratings, and reviews
Nov 26–Dec 5	Streaming availability and recommendations; integrate core features
Dec 6–12	Test full user workflows, fix defects, refine interface
Dec 13–15	Deployment checks, sample accounts, demo rehearsal

7. Challenges and Feasibility

External API integration: Catalog requests may fail, and streaming availability changes and varies by country. We will verify API access early, cache frequently requested data where permitted, and show clear messages when availability data is missing.

Recommendation quality: New users have little history, and viewing frequency alone may not reflect taste. We will collect favorite genres up front, weight ratings more heavily, and start with an explainable scoring system tested on sample viewing histories.

Data consistency and access control: Library statuses, episode progress, and reviews must stay accurate and tied to the right account. We will validate requests on the server and test persistence, progress calculations, and attempts to modify another user's records.

Feasibility: Established APIs supply the catalog data, so our team can focus on the interface, persistent user data, and recommendation logic. We will prioritize the five core features and reserve the final development period for integration, testing, and demo preparation.